

Architecting reproducible science: Best Scientific Computing Practices

From Exploratory Notebooks to Reliable Scientific Software

Fernando Garzon, August 2026

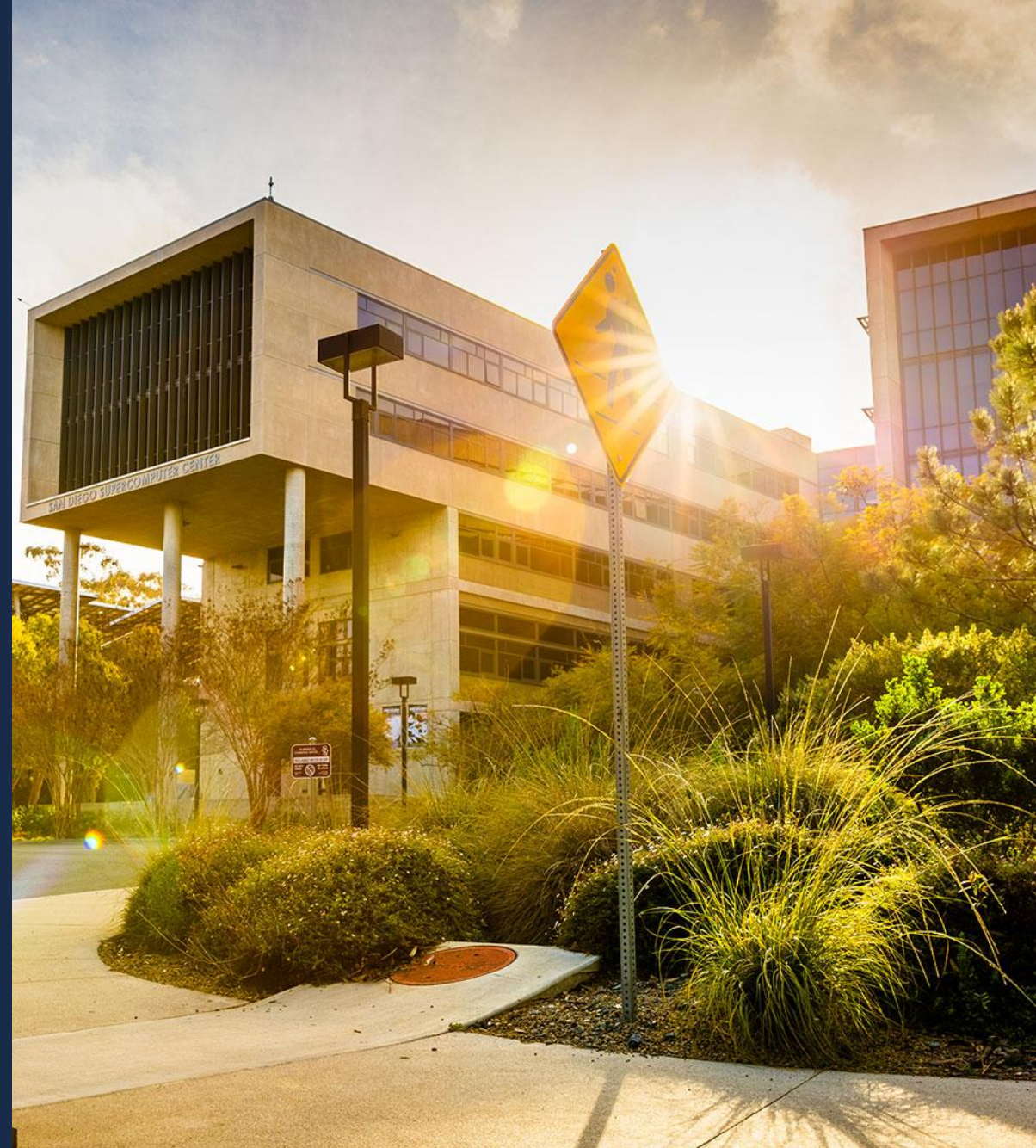
[linkedin.com/in/fgarzonm](https://www.linkedin.com/in/fgarzonm) - github.com/ofgarzon2662

SDSC
SAN DIEGO SUPERCOMPUTER CENTER

UC San Diego

Why Do I care about this?

- Research Software Engineer at SDSC
- Support scientific workflows across HPC and cloud environments
- Build research software, infrastructure, and automation.
- I've seen magnificent programmers fail because they insisted on following poor software engineering practices.
- You may write brilliant algorithms. But if nobody can reliably run, test, or reuse your code, its impact `=== null`;
- My goal today is to show you how simple software engineering practices can turn brilliant scientific code into a reliable, reproducible workflow.

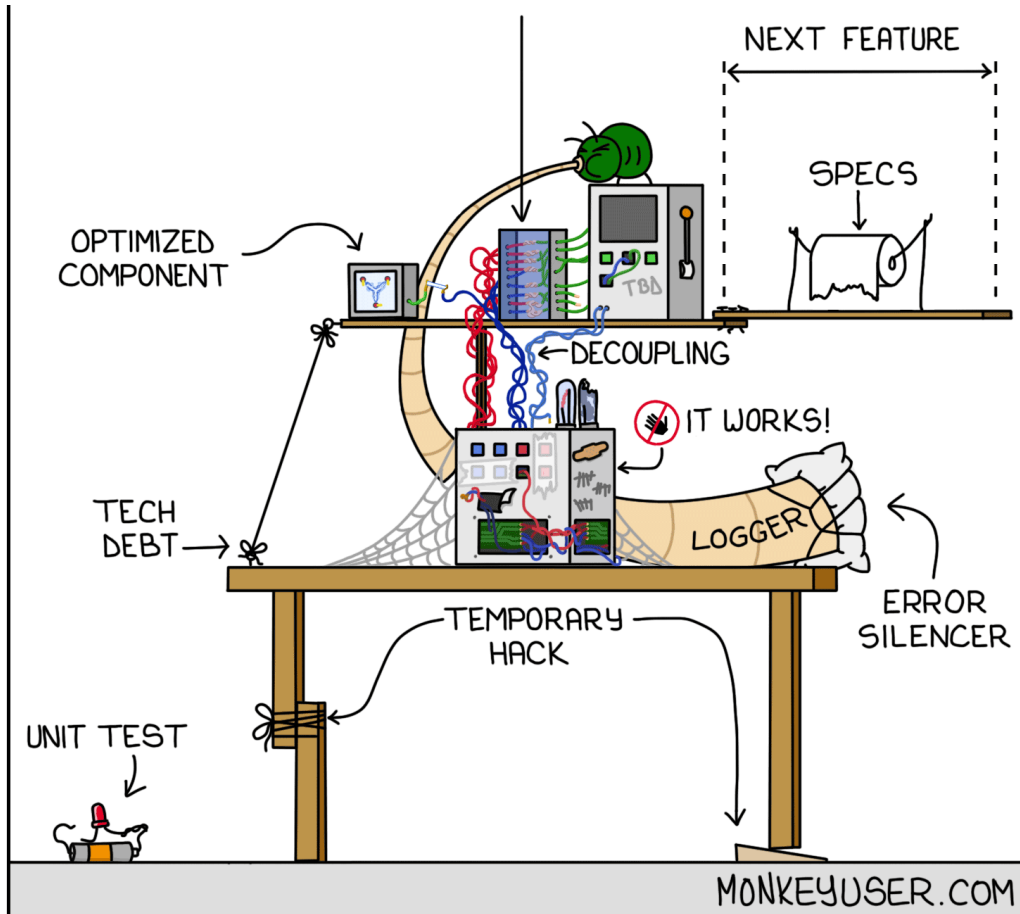




Outline

1. But It Worked on My Laptop: The Horror Story
2. Reproducible Code $\neq$ Reproducible Workflow; `// true`
3. Great for Exploration. Fragile as a Workflow.
4. Let's Unpack This.
5. AI-Driven Development—Let's Do It Right.
6. Tiny Tests, No Regrets.
7. Take Big Action.
8. MLOps.
9. Final Remarks.

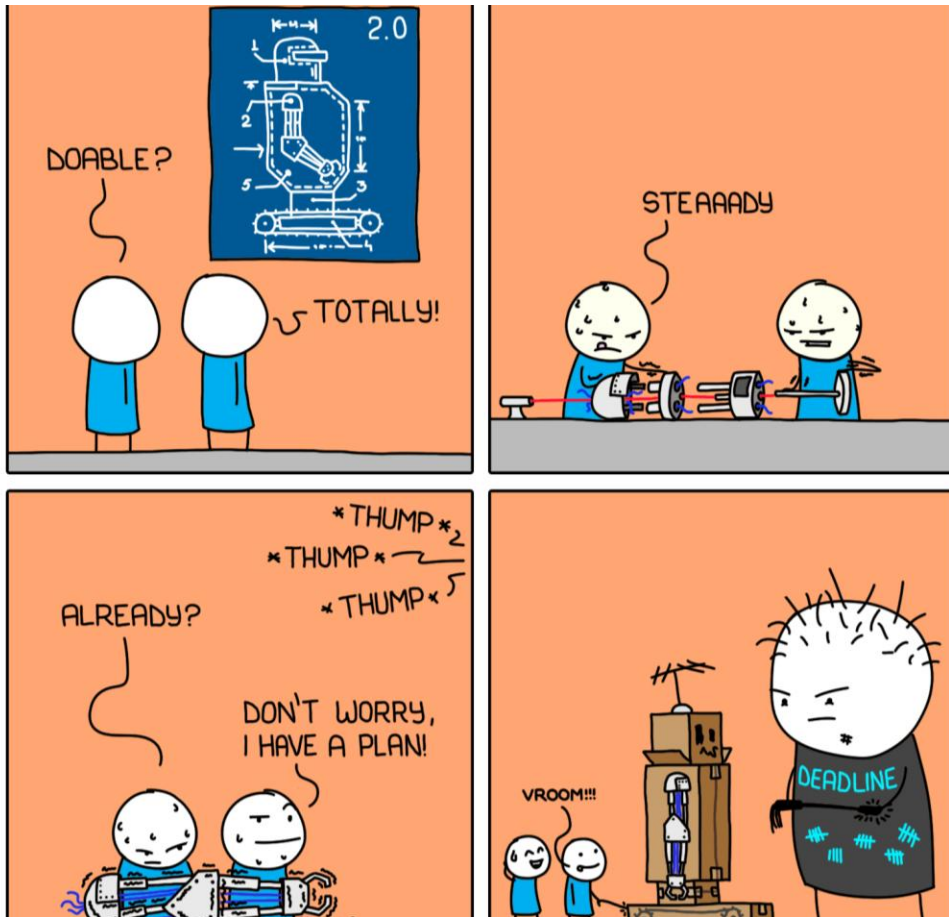
But It Worked on My Laptop: The Horror Story



- Milena's model in the NB worked perfectly. It ingested and analyzed the data, generated the proper outputs, and most importantly, it promised a big balance in her bank account.
- The mess up: She forgot to run an apparently meaningless cell, one that optimized the routine to prevent memory overload. Fans roared and laptop froze.
- Results: Terrible. Stakeholders who care nothing about code and all about results decided this isn't worth their time.
- The model was correct. The code was there. So what was missing?
- ***What did she actually need to reproduce the good results she had previously published?***

<https://www.youtube.com/watch?v=5AK7SKgkjms>

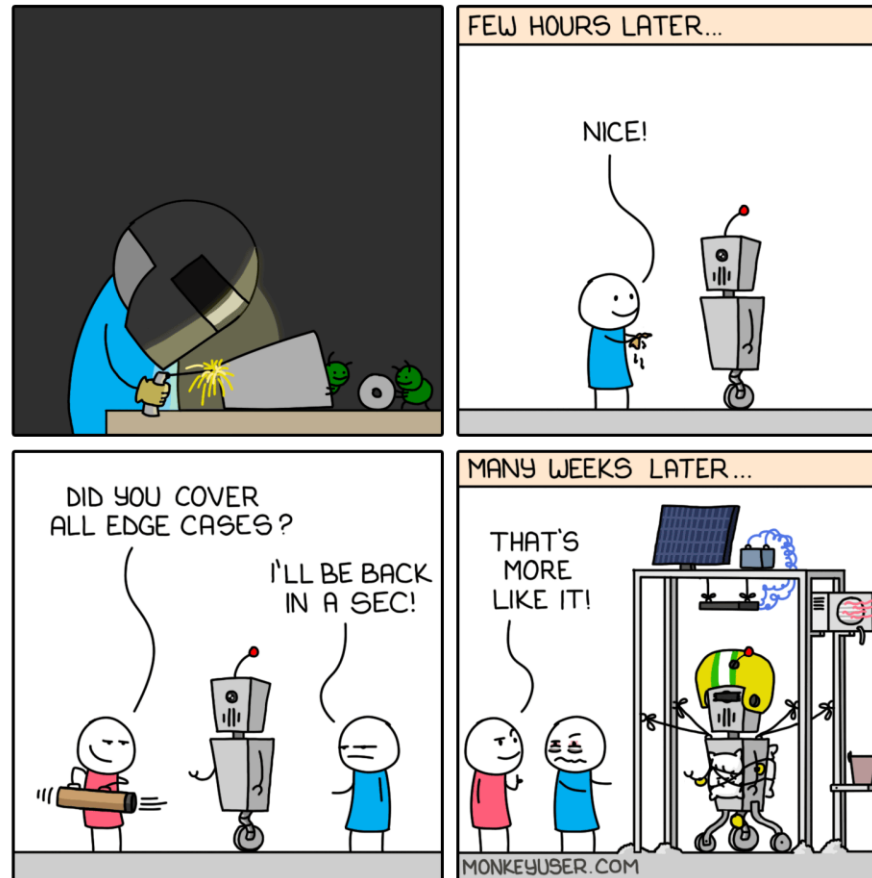
Reproducible Code != Reproducible Workflow; // true



- The code was perfectly preserved on the NB, but successful execution still depended on Milena's memory.
- Manual process that depends on someone running every cell in the correct order is highly prone to error.
- The more a workflow depends on undocumented human decisions, the harder it is to reproduce reliably.
- Mature scientific & non-scientific software turns those decisions into an explicit and predictable execution process.
- **For the complete workflow to be reproducible, it should be:**
 - **Explicit:** Are the inputs, dependencies, configuration, and execution order clearly defined?
 - **Portable:** Can the workflow run in a clean environment—not only on the original laptop?
 - **Testable:** Can individual components be verified without running the entire notebook?
 - **Automated:** Can the complete workflow run correctly without someone remembering every step?

Great for Exploration. Fragile as a Workflow.

EDGE CASES



- NBs are amazing for exploratory work: You got an idea, you test it on the fly, you see instant results/plots/numbers just one 'shift' + 'enter' hit away.
- However, and for the *same* reason, if good coding practices aren't kept (and *most* people in academia don't), NBs can grow out of control quickly with messed up code all over the cells: hidden state, duplicated logic, out-of-order execution, etc.
- And AI just facilitates the creation of big messed up code-bases faster.
- So yeah, NBs are great, **but their workflows are fragile and hard to replicate.**
- The solution?

Modular Python Packages

- Well, *More precisely: Move stable scientific logic into a modular Python package.*
- and keep the Notebook, it's still valuable.

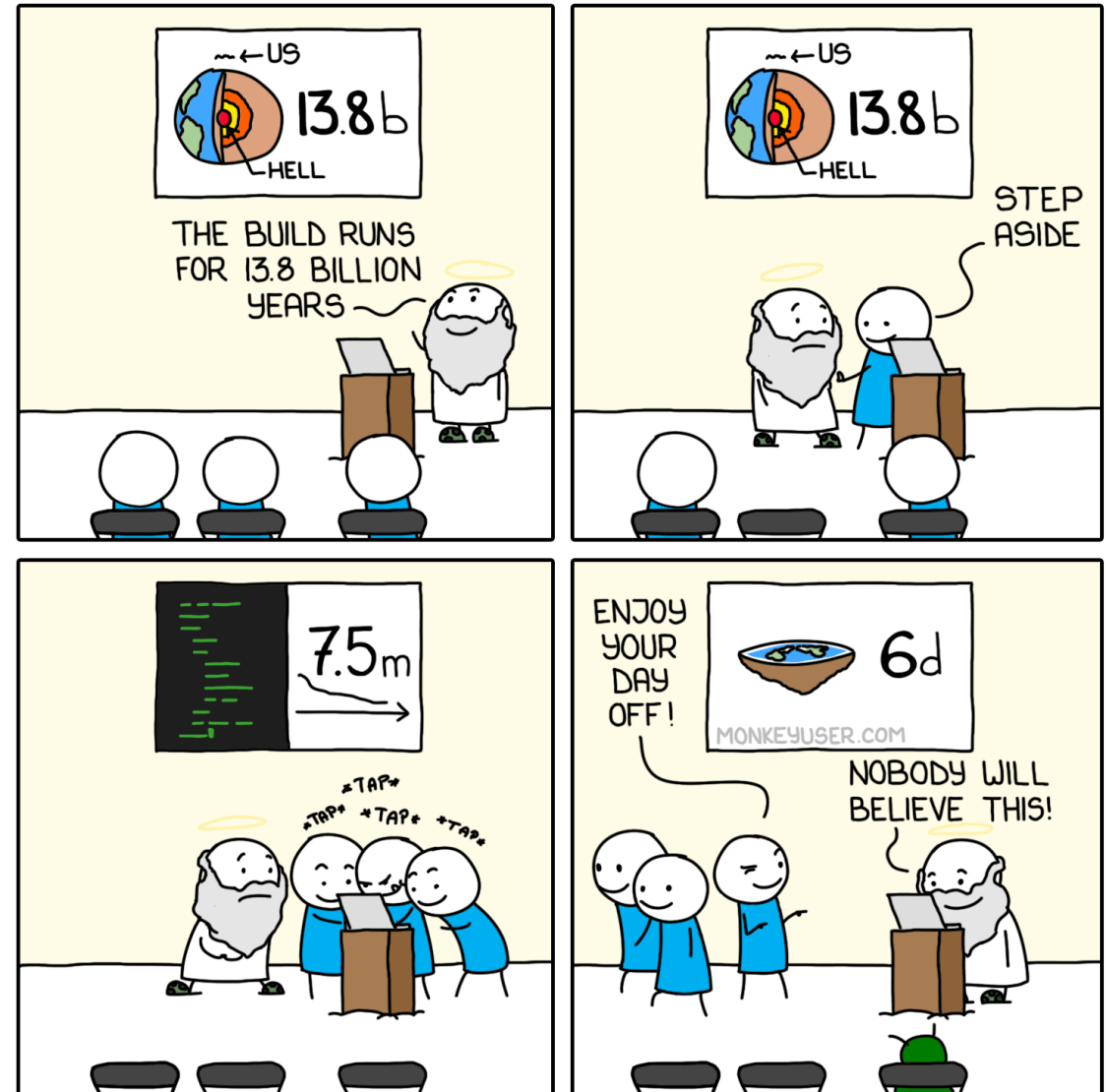
Let's Unpack This.

- Modular Python packages give stable scientific logic a clear, reusable structure.
- **IF** coded properly, they are small, manageable, testable, reusable, and easier to deploy across different environments.
- Think of them as individual bricks: each component is easier to understand, test, replace, and reuse than the entire building.
- Packages do not make an application scalable by themselves, but they provide the modular foundation needed to build reliable and scalable systems.
- Therefore, our next step is clear: move the stable scientific logic out of the notebook and into a modular Python package.

Follow the instructions in here:

https://github.com/ofgarzon2662/mnist_ae/blob/master/README.md

NEW WORLD



AI-Driven Development—Let's Do It Right

Prompt Engineering and AI assisted development. Use SOLID principles:

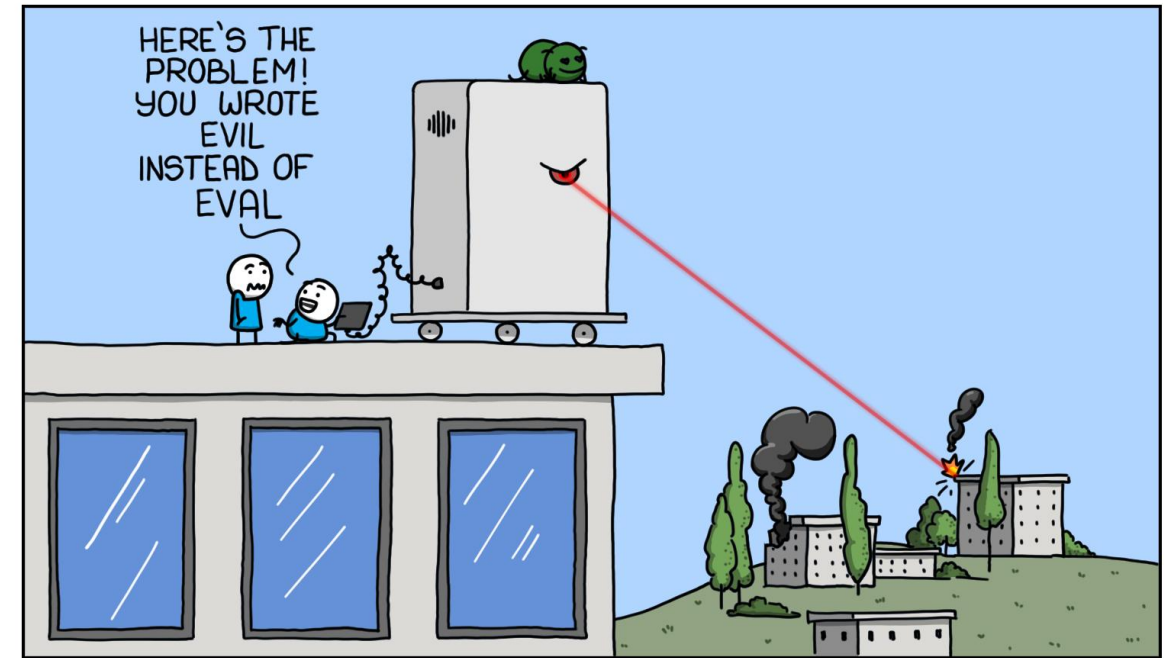
Take a look at

https://github.com/ofgarzon2662/mnist_ae/blob/master/README.md
`#inspect-what-nbdev-generated`

- If good coding practices aren't kept, AI is just an accelerator of bad systems *wrapped* in hyper-polished code.
- Clear, specific, and rigorous instructions are needed. AI replaces the typing, but the System Design practices will always belong to the human in the loop.
- Claude Code won't get fired for implementing the wrong workflow,
- ...but you might

NATURAL LANGUAGE INSTRUCTIONS

MONKEYUSER.COM

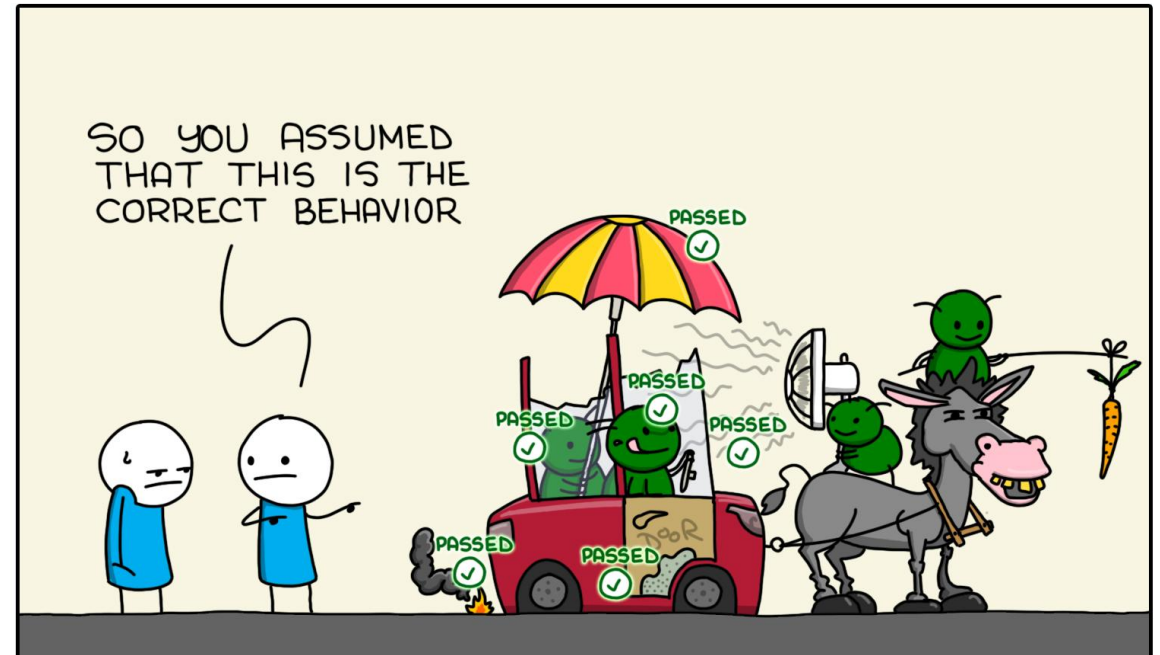


Tiny Tests, No Regrets / No Tests, No Gains.

- Deploying the complete workflow on Expanse or in the cloud just to discover whether it works is slow, expensive, and painful.
- Unit tests let you verify small pieces of scientific logic quickly *before* requesting large computing resources.
- Packaging the stable logic makes it easier to isolate, test, and reuse.
- Unit testing should be taught in every programming course and adopted from the beginning of every serious software project.
- Unit tests are not the only tests we need, but they catch bugs early when they are easier and cheaper to fix.

UNIT TESTS

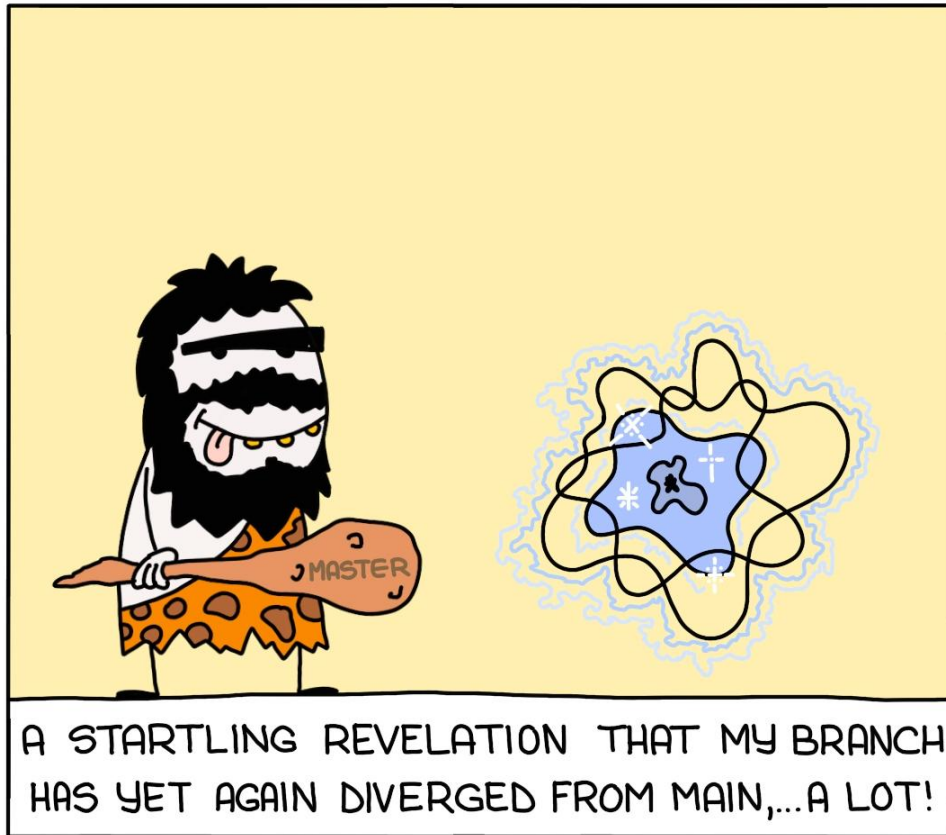
MONKEYUSER.COM



Take Big Action.

RAMIFICATIONS

MONKEYUSER.COM



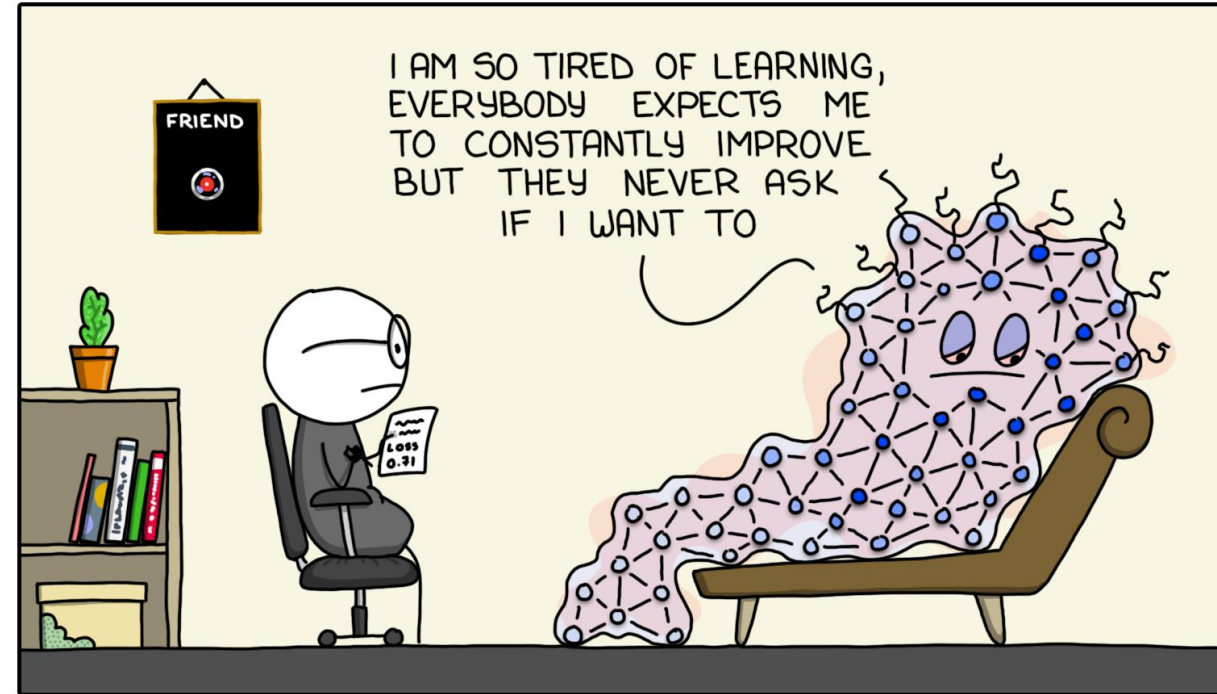
- DevOp\$ is a seriously promising profitable field of work. And it's been for the last 15 years.
- Combining DevOps with AI tools can make you super attractive to potential employers in high tech industries and HPC.
- DevOps is the art of automating deployments in a fast, controlled, and a reproducible manner.
- **GitHub Actions** are a great first step on the DevOps journey.
- They're easy to implement, free or inexpensive, and should be included in every project you work on—starting today.

MLOps: Turning your Notebook into a Money-Printing Machine.

- MLOps applies DevOps practices to machine-learning systems.
- It makes code, data, models, tests, releases, and monitoring more repeatable.
- The goal is reliable model behavior from experiment through deployment.
- Your ML expertise becomes much more useful when you can test, version, deploy, and operate the system around the model.
- Those are concrete, portfolio-ready skills that employers recognize.

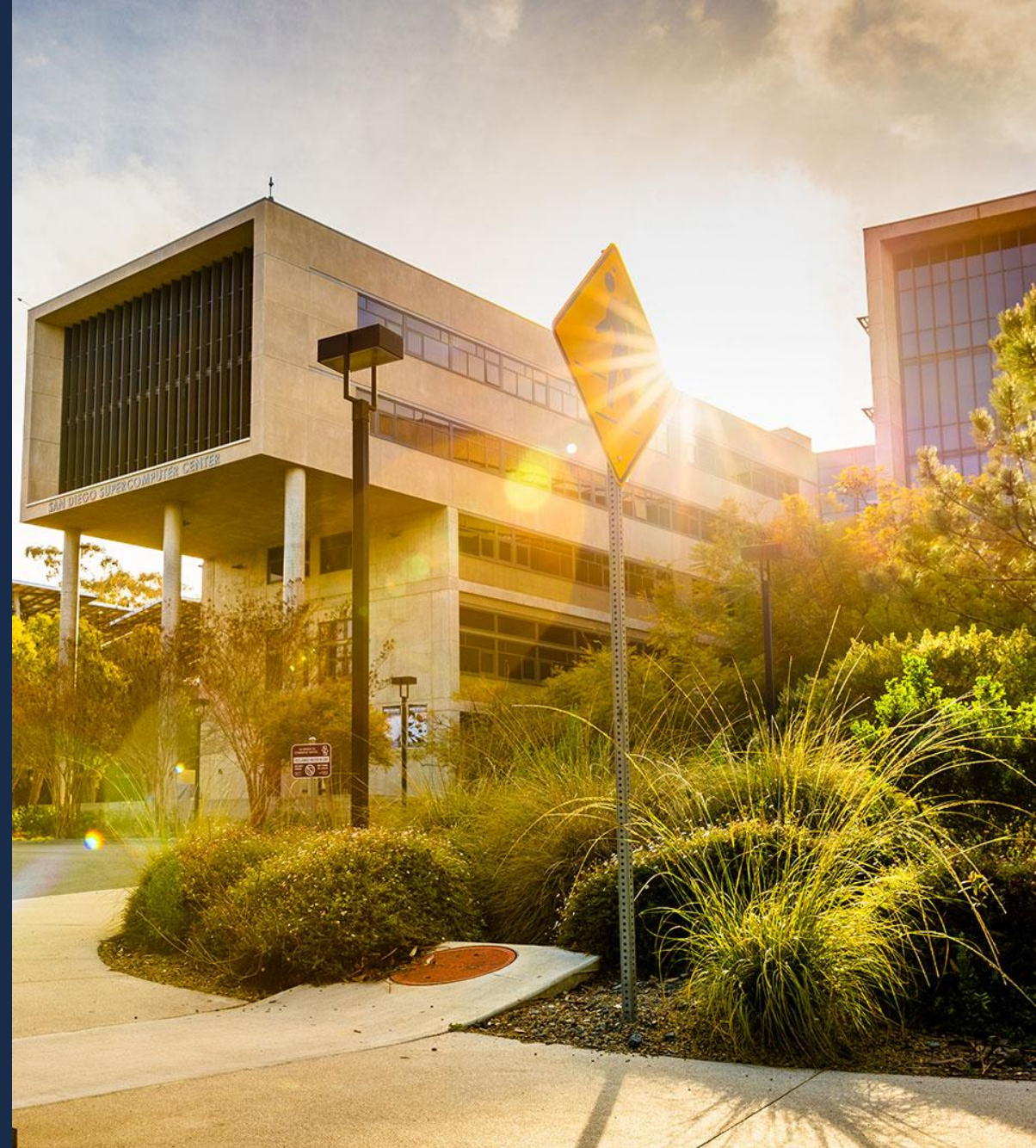
AI CARE

MONKEYUSER.COM



Final Remarks

1. There are bad ML/AI Engineers, and then there are ML/AI Engineers who can deploy their code to prod.
2. Don't commit big resources and time into testing ML/AI models. Use automated Unit Test first.
3. Github Actions are the 1st step in the automation process but there is plenty more you can learn to leverage your computing practices: (CI/CD, DevOps, MLOps).
4. AI won't take your AI Engineering job. Someone who knows how to use DevOps pipelines to streamline the deployment of AI/ML models into production, that person will.
5. Reproducibility is not just preserving the code. It is preserving—and automating—the complete workflow required to produce the result.





Questions



[linkedin.com/in/fgaronm](https://www.linkedin.com/in/fgaronm)



github.com/ofgarzon2662